RESUME PROJECT · BUILD GUIDE

RepoSage

A code-aware Retrieval-Augmented Generation system that answers natural-language questions about any GitHub repository — with file/line citations and a real evaluation harness to prove it works.

FastAPI · tree-sitter · Qdrant · React · GCP Cloud Run · RAG · LLM-as-Judge Eval

---

A step-by-step plan designed to be built **with Claude Code**.

Prepared for **Nani · Kurapati Pvt Ltd** · ~4 weeks, part-time

What's Inside

This guide takes you from an empty folder to a deployed, demo-ready project. Each section is sequenced so you always have something working before you add the next layer.

1 · The Pitch & Why It Stands Out

RepoSage lets a user point at a GitHub repository and ask plain-English questions like *"where is authentication handled?"* or *"what does the payment module depend on?"* It answers using the actual code, and every answer links back to specific files and line ranges.

Most RAG resume projects are "chat with your PDF" clones. Recruiters have seen hundreds of them. RepoSage is different because of one core decision:

> **THE IDEA THAT MAKES IT STRONG**
>
> Naive RAG splits text every N characters. That **destroys code** — it cuts functions in half and separates a function from its docstring. RepoSage chunks by **syntax tree** (using tree-sitter), so every chunk is a complete function, class, or method with its metadata intact. That single sentence is your headline interview moment.

On top of that, you'll add an **evaluation harness** that measures whether retrieval is actually good (recall@k, MRR, faithfulness). Almost no fresher project measures quality — being able to say *"adding re-ranking improved recall@5 from 0.68 to 0.84"* signals engineering maturity, not just the ability to follow a tutorial.

What you'll be able to claim

- Built a real RAG pipeline — not an API wrapper — with a defensible design decision (AST chunking).
- Deployed it to the cloud using managed services (containerized backend + managed vector DB).
- Measured and improved retrieval quality with proper metrics.
- Shipped it full-stack with a usable frontend.

2 · How the System Works

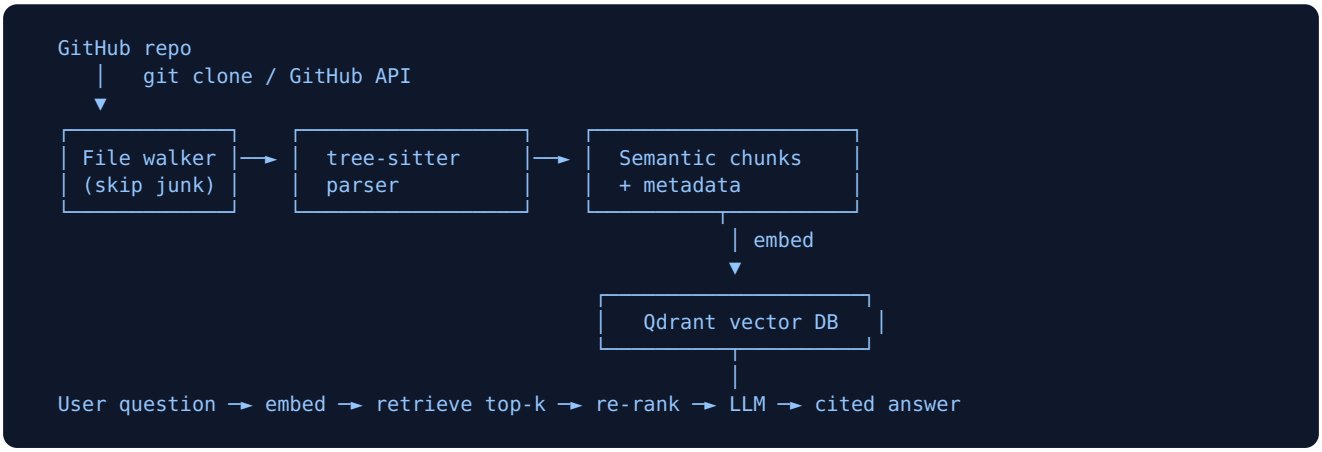
There are two phases. **Ingestion** happens once per repo (and again when code changes). **Query** happens every time a user asks something.

Phase A — Ingestion

1. Clone the target repo (via the GitHub API or `git clone`).
2. Walk the file tree, skipping junk (`node_modules`, `dist`, `.git`, binaries).
3. Parse each source file with **tree-sitter** into a syntax tree.
4. Extract chunks at function / class / method level, each carrying metadata: file path, start line, end line, language, parent class.
5. Convert each chunk into an embedding (a vector) using a code-aware embedding model.
6. Store vectors + metadata in the **Qdrant** vector database.

Phase B — Query

1. User asks a question. Embed the question into the same vector space.
2. Retrieve the top-k most similar code chunks from Qdrant.
3. (Optional) Re-rank those chunks with a cross-encoder for better ordering.
4. Send the question + retrieved code (with metadata) to the LLM.
5. Return the answer with clickable citations like `src/auth/login.py#L42-L88`.



> **MENTAL MODEL**
>
> Think of it as a librarian. Ingestion = reading every book and writing index cards (chunks). Query = the librarian finds the right cards, reads them, and answers in their own words while pointing at the exact shelf (citation).

3 · Tech Stack & Accounts

Layer	Choice	Why
Backend	FastAPI (Python)	Clean async API, natural fit for ML/Python code
Parsing	tree-sitter	Grammars for Python, JS, Java, Go — your "multi-language" claim
Embeddings	Code-aware model (API or open model)	Code embeddings beat generic text ones on code
Vector DB	Qdrant (free cloud tier)	First-class metadata filtering, generous free tier
LLM	Anthropic / OpenAI API	Answer generation from retrieved context
Frontend	React + Tailwind	Chat UI with code blocks + citation links
Backend host	GCP Cloud Run	Containerized, scales to zero, cheap
Frontend host	Vercel	Free, instant React deploys

Accounts to create before you start

- **Qdrant Cloud** — free cluster, grab the URL + API key.
- **LLM provider** — Anthropic or OpenAI API key. Set a spend limit.
- **GitHub** — a personal access token (read-only) for cloning repos via API.
- **Google Cloud** — for Cloud Run (free credits for new accounts).
- **Vercel** — for the frontend (sign in with GitHub).

PROTECT YOUR KEYS

Never hard-code API keys. Put them in a `.env` file, add `.env` to `.gitignore` on day one, and set spend limits on every paid API. A leaked key in a public repo can cost real money.

4 · Working with Claude Code

Claude Code is a terminal tool that writes, edits, and runs code in your project. The difference between a frustrating session and a smooth one is how you scope your requests. Here's how to drive it well.

Golden rules

- **One milestone per request.** Don't say "build the whole app." Say "build the tree-sitter chunker for Python and write a test that proves it produces one chunk per function."
- **Always ask for a test or a way to verify.** If you can't run it and see it work, you don't actually have it.
- **Give it context.** Paste errors in full. Tell it what you already have. Reference file names.
- **Review before you accept.** Read what it wrote. You need to understand your own project for the interview — that's the whole point.
- **Commit often.** After each working milestone, `git commit`. If the next step breaks things, you can roll back.

SET UP A PROJECT MEMORY FILE

On your very first run, ask Claude Code to create a `CLAUDE.md` file in the repo root describing the project, stack, and conventions. Claude Code reads it automatically each session, so you don't repeat yourself. Keep it updated as decisions change.

Throughout this guide, dark boxes labelled **PROMPT** are starting prompts you can adapt and paste into Claude Code. They're deliberately specific — vague prompts get vague code.

PROMPT · FIRST SESSION

```
Create a CLAUDE.md for a project called RepoSage: a code-aware RAG system that ingests a GitHub repo, chunks code by AST using tree-sitter, embeds chunks into Qdrant, and answers questions with file/line citations via an LLM. Backend is FastAPI, frontend is React+Tailwind. Set up a Python project with a virtual environment, a sensible folder structure (ingestion/, retrieval/, api/, eval/), a requirements.txt, and a .gitignore that excludes .env and venv. Then explain the folder layout to me.
```

5 · Week 1 — The RAG Core

This is the hard part and the heart of the project. Do not move on until retrieval works locally. Everything else is comparatively easy.

Milestone 1.1 — Repo ingestion

Get code from a repo onto disk and walk the files, skipping the junk.

PROMPT

```
Write an ingestion module that takes a GitHub repo URL, clones it to a temp directory, and walks all files. Skip directories like node_modules, .git, dist, build, venv, and skip binary/non-code files. Return a list of (file_path, language, source_code) for supported languages, starting with Python. Add a quick script I can run on a small public repo to print how many files it found.
```

Milestone 1.2 — AST chunking (the centerpiece)

This is the decision that makes RepoSage special. Get it right.

PROMPT

```
Using tree-sitter for Python, write a chunker that parses each source file and extracts one chunk per top-level function and per class method. Each chunk must include: the code text, file path, start line, end line, language, and parent class name if any. If a single function is larger than ~150 lines, split it into overlapping sub-chunks. For files tree-sitter can't parse, fall back to line-based chunking with overlap. Write a test on a sample file that asserts each function becomes its own chunk with correct line numbers.
```

WHERE TIME DISAPPEARS

Walking the tree-sitter parse tree and getting line numbers exactly right is the fiddliest part of the whole project. Budget extra time here. If you're stuck after a day, ship a simpler version (functions only, no sub-chunking) and come back later — a working simple chunker beats a perfect broken one.

Milestone 1.3 — Embeddings + Qdrant

PROMPT

```
Write a module that takes my code chunks, generates embeddings using [your chosen embedding model], and upserts them into a Qdrant collection along with their metadata (file path, line range, language). Batch the embedding calls so I don't make one request per chunk. Read the Qdrant URL and API key from .env. Add a script that ingests a small repo end-to-end and prints the collection size.
```

Milestone 1.4 — Retrieval

PROMPT

```
Write a retrieval function: given a text question, embed it and query Qdrant for the top-k most similar chunks, returning their code and metadata. Make a CLI script where I type a question and see the top 5 retrieved chunks with their file paths and line numbers. Test it against the repo I already ingested.
```

END-OF-WEEK-1 SUCCESS TEST

You ingest a real repo, type a question on the command line, and see relevant functions come back with correct file paths and line numbers. No API, no UI yet — just proof the brain works. Commit this.

6 · Week 2 — API + Frontend

Goal: a working end-to-end demo in the browser by the end of the week.

Milestone 2.1 — LLM answer generation

PROMPT

```
Write a function that takes a question and the retrieved code chunks, builds a prompt instructing the LLM to answer ONLY from the provided code and to cite sources as file_path#Lstart-Lend, and calls the LLM API. Return the answer text plus the list of cited chunks. If the retrieved code doesn't contain the answer, the model should say so rather than guess.
```

Milestone 2.2 — FastAPI endpoints

PROMPT

```
Wrap the pipeline in FastAPI. Endpoints: POST /ingest (takes a repo URL, runs ingestion, returns a job status) and POST /query (takes a question, runs retrieval + LLM, returns answer + citations). Add CORS so a React frontend can call it. Include a /health endpoint. Show me how to run it locally and test with curl.
```

Milestone 2.3 — React chat UI

PROMPT

```
Build a React + Tailwind frontend: an input to submit a GitHub repo URL (calls /ingest with a loading state), and a chat interface to ask questions (calls /query). Render answers with proper code-block formatting, and render each citation as a clickable link to the file/line on GitHub. Keep it clean and minimal. Point it at my local FastAPI server.
```

END-OF-WEEK-2 SUCCESS TEST

In the browser: paste a repo URL, wait for ingestion, ask a question, get a cited answer with working GitHub links. That's a complete, demoable product. Record a short screen capture now — it's your backup if deployment gets messy later.

7 · Week 3 — Evaluation & Re-ranking

This is what lifts the project from "nice clone" to "this person measures their work." Keep it focused — a small, honest eval beats a sprawling one.

Milestone 3.1 — Build a labeled test set

Pick one well-known repo. Write 15–25 questions, and for each, note which file(s) *should* be retrieved to answer it. Store as a simple JSON or CSV.

PROMPT

```
Help me build an evaluation set for RepoSage. Create a JSON schema where each entry has: a question, the list of file paths that contain the correct answer, and an optional ideal answer. Then suggest 15 diverse questions for [chosen repo] covering architecture, specific functions, dependencies, and "where is X handled" style queries. I'll fill in the correct file paths.
```

Milestone 3.2 — Retrieval metrics

PROMPT

```
Write an eval script that runs every question in my test set through retrieval and computes recall@k (did we retrieve at least one correct file in the top k?) and MRR (mean reciprocal rank of the first correct hit). Print a summary table and save results to a file so I can compare runs.
```

Milestone 3.3 — Faithfulness (LLM-as-judge)

PROMPT

```
Add a faithfulness check: for each generated answer, use a separate LLM call as a judge to score whether the answer is fully supported by the retrieved code (no hallucination), returning a 1-5 score and a one-line reason. Aggregate the average score across the test set.
```

Milestone 3.4 — Add re-ranking and measure the gain

PROMPT

```
Add an optional re-ranking step: retrieve a larger candidate set (e.g. top 20) from Qdrant, then re-rank with a cross-encoder and keep the top 5. Make it toggleable. Then run my eval set with and without re-ranking and show me a before/after comparison of recall@5 and MRR.
```

THIS IS YOUR MONEY QUOTE

Whatever the real numbers turn out to be, the before/after comparison is what you'll put on your resume and lead with in interviews. Record the exact numbers. Never round them up or invent them — a sharp interviewer will ask precisely how you measured, and your honest answer is the impressive part.

8 · Week 4 — Deploy & Polish

Make it real, make it look good, make it explain itself.

Milestone 4.1 — Containerize & deploy backend

PROMPT

```
Write a Dockerfile for the FastAPI backend and walk me through deploying it to Google Cloud Run, including how to set my environment variables (Qdrant URL/key, LLM key, GitHub token) as Cloud Run secrets rather than baking them into the image. Give me the exact gcloud commands.
```

DO THE SECRETS YOURSELF

Claude Code can write the commands, but **you** should be the one to paste in actual API keys and run the deploy. Don't put real secrets into files or chat. Set them as managed secrets / environment variables in the Cloud Run console.

Milestone 4.2 — Deploy frontend

PROMPT

```
Help me deploy the React frontend to Vercel and point its API base URL at my deployed Cloud Run backend via an environment variable. Make sure CORS on the backend allows the Vercel domain.
```

Milestone 4.3 — README & demo

PROMPT

```
Write a polished README.md: one-line description, the architecture diagram (ASCII or link to an image), the key design decision (AST chunking and why), the tech stack, setup instructions, and a results section with my evaluation numbers. Keep it recruiter-skimmable with clear headers.
```

- Record a 30–60 second demo GIF of the live app and embed it at the top of the README.
- Make sure the repo is public and the live link works.
- Pin the repo on your GitHub profile.

END-OF-WEEK-4 SUCCESS TEST

A stranger can click your live link, ask a question about a repo, and get a cited answer — and your README explains how it works and how good it is, with numbers.

9 · The Tricky Parts

These are where projects like this stall. Knowing them in advance is half the battle — and each one is also great interview material.

AST chunking granularity

A 500-line class shouldn't be one chunk; a 3-line helper shouldn't be split. You need a size-aware strategy: chunk at function/method level, sub-chunk oversized functions with overlap, and fall back to line-based chunking for unparseable files. Start simple (functions only) and refine.

Large repositories

Real repos have thousands of files; you can't embed everything on a free tier. Aggressively skip vendored/generated directories, filter by file extension, and batch your embedding calls. Mentioning that you handled scale shows production awareness.

Citations that actually resolve

The metadata (file path + line range) must thread cleanly from chunk → retrieval → answer → clickable GitHub link. It's fiddly but it's the single most impressive thing in a live demo, so it's worth the effort.

Incremental re-indexing (stretch goal)

Re-embedding a whole repo on every change is wasteful. If you have time, hash file contents and only re-embed changed files. Strong "I thought about scale" signal — but genuinely optional. Cut this before you cut the eval harness.

> **SCOPING PRIORITY IF YOU RUN SHORT ON TIME**
>
> Protect the evaluation harness above all — it's what makes the project memorable. Cut in this order: (1) incremental re-indexing, (2) extra languages beyond Python + one more, (3) re-ranking. Never cut the eval.

10 · Interview Talking Points

You built it so you could understand it. Be ready to discuss:

- **Why AST chunking?** "Fixed-size chunking splits functions and separates them from their context, so retrieved chunks were semantically meaningless. Chunking by syntax tree keeps each function whole, which improved retrieval quality measurably."
- **How did you know it improved?** "I built a labeled eval set and measured recall@k and MRR. Re-ranking took recall@5 from X to Y." (Use your real numbers.)
- **How did you handle hallucination?** "The prompt constrains the model to answer only from retrieved code, and I scored faithfulness with an LLM-as-judge over my test set."
- **How would you scale it?** "Incremental re-indexing by file hash, async ingestion jobs, and metadata filtering to scope retrieval per-repo."
- **What would you do differently?** Have an honest answer ready — interviewers love this one.

Resume bullet (swap in your real numbers)

> **REPOSAGE — CODE-AWARE RAG FOR CODEBASE Q&A**
>
> *FastAPI, tree-sitter, Qdrant, React, GCP Cloud Run*
>
> Built a retrieval-augmented system answering natural-language questions about any GitHub repo with file/line citations. Chunked code by AST (function/class level) instead of fixed-size splits to preserve code semantics across multiple languages. Added a re-ranking step and an evaluation harness (recall@k, MRR, LLM-judged faithfulness), improving recall@5 from 0.68 to 0.84.

Replace 0.68 → 0.84 with whatever you actually measure. Fabricated numbers fall apart under one good follow-up question; real ones — even modest ones — are far more convincing.

11 · Master Checklist

Week 1 — Core

- [ ] Project scaffolded, `CLAUDE.md` + `.gitignore` + `.env` in place
- [ ] Repo ingestion walks files and skips junk
- [ ] tree-sitter chunker produces one chunk per function with correct line numbers
- [ ] Chunks embedded and stored in Qdrant
- [ ] CLI retrieval returns relevant chunks for a typed question

Week 2 — Product

- [ ] LLM generates cited answers from retrieved code
- [ ] FastAPI `/ingest` and `/query` endpoints working
- [ ] React chat UI with clickable citations
- [ ] End-to-end demo working locally + screen recording saved

Week 3 — Rigor

- [ ] Labeled eval set (15–25 questions) built
- [ ] recall@k and MRR computed
- [ ] Faithfulness scored via LLM-as-judge
- [ ] Re-ranking added; before/after numbers recorded

Week 4 — Ship

- [ ] Backend containerized and deployed to Cloud Run (secrets set safely)
- [ ] Frontend deployed to Vercel, pointing at live backend
- [ ] README with architecture, design rationale, and eval numbers
- [ ] Demo GIF embedded; repo public and pinned

> **FINAL REMINDER**
>
> Commit after every working milestone. A working simple version beats a broken ambitious one — and you can always come back to add depth. Good luck, Nani.